MambaNIDS: Robust Real-Time Cyberattack Detection via Selective State-Space Modelling and Frequency-Domain Attention

Sufyan Jabbar Ahmed Al-Ezzi*[1]

[1]Department of Computer Science, University of Baghdad, Baghdad, Iraq

Abstract

Modern deep-learning network intrusion detection systems (NIDS) face two unsolved tensions: (i) recurrent backbones such as Bi-LSTM are $O(L)$ but sequentially constrained, while Transformer detectors offer parallel attention at $O(L^2)$ memory cost; and (ii) high-accuracy detectors frequently degrade catastrophically under adversarial flow perturbations and are reported with single-seed point estimates rather than statistically grounded confidence intervals. We propose MambaNIDS, a network intrusion detection architecture built on selective state-space models (Mamba). MambaNIDS couples a hierarchical 1D-CNN front end with a two-layer Mamba residual stack and a novel dual-attention head that fuses time-domain Multi-Head Attention with a Frequency-Domain Attention (FDAttn) branch via a learnable per-token gate. We carry out a publication-grade evaluation against five baselines (LSTM, GRU, CNN-LSTM, CNN-BiLSTM-MHA, and Transformer-IDS) on the TON_IoT Network dataset (211,043 flows over ten classes), and cross-validate on N-BaIoT and IoTNet24. Averaged over three independent random seeds, MambaNIDS achieves the highest mean macro F1 on TON_IoT ($88.94 \pm 0.66\%$) with 1.63 M parameters — 60% fewer than CNN-BiLSTM-MHA — while sustaining 5.86 ms inference latency per batch of 512 flows. The same architecture and hyperparameters generalise to N-BaIoT (89.62% accuracy) and IoTNet24 (99.62% accuracy) without modification. Under Madry-style adversarial training the proposed model retains 69.66% FGSM and 68.75% PGD-10 accuracy at perturbation budget $\epsilon = 0.05$ — a 2.8×

*Corresponding author: sufianjabbar1@gmail.com. Department of Computer Science, University of Baghdad, Baghdad, Iraq (added in revision)

improvement over the clean-trained model at the same budget. Beyond the architectural and robustness contributions, we derive the first non-vacuous McAllester PAC-Bayes generalisation bound reported for a deep NIDS, conduct a controlled three-way component ablation, and provide a SHAP-based feature attribution audit confirming that the detector relies on semantically meaningful flow signals rather than dataset shortcuts. The complete evaluation pipeline — pure-PyTorch Mamba, Holm–Bonferroni-corrected McNemar tests, Cohen's d effect sizes, a complexity Pareto analysis, and a PAC-Bayes bound estimator — is released alongside trained checkpoints to support replication.

Keywords: Intrusion detection · Selective state-space models · Mamba · Frequency-domain attention · Adversarial robustness · PAC-Bayes · Statistical testing · IoT security

Response to reviewers (revision): this version addresses the reviewer and editorial comments in full. Red-coloured insertions throughout the manuscript identify every location that was changed. In summary: (i) the author affiliation is now supplied; (ii) the novelty statement is narrowed and concurrent 2025 Mamba-based IDS work is cited in Sections 2.3 and 7.3; (iii) full per-baseline hyperparameters, including CNN-BiLSTM-MHA, are supplied in Section 6.3; (iv) leakage-free chronological / device / capture-interval splitting is documented in Section 6.2; (v) the multi-seed protocol scope is clarified in Section 6.4; (vi) the adversarial-robustness claims are recomputed and split by attack type in Section 7.5 and rewritten in Sections 8–9; (vii) Table 5 dashes for LSTM/GRU are replaced by numeric entries reconciled with Fig. 4.

1 Introduction

The continued proliferation of heterogeneous IoT devices has widened the network attack surface faster than defenders can adapt: recent industry reports place the year-on-year growth of IoT-targeted attacks above 30% [1]. Signature-based intrusion detection systems (NIDS) such as Snort and Suricata [2] cannot generalise to zero-day or polymorphic attacks, leaving data-driven detectors as the only viable defence at line speed [3]. Yet not all data-driven architectures meet the dual constraint of real-time inference and statistical reliability that production deployment demands; three families currently dominate the literature, each with distinct limitations.

Recurrent backbones (LSTM, GRU) process flow sequences in $O(L)$ time but their dependency graphs prevent parallel computation across the temporal dimension [4,5].

Convolutional backbones (1D-CNN, CNN-LSTM) capture local patterns extremely well and are highly parallelisable, but their receptive fields are limited and they ignore long-range temporal dependencies unless stacked deeply [6, 7].

Attention-based backbones (Transformer-IDS, GRU-Attention) offer global interactions but at quadratic memory and time cost [8–10]. Across all three families no work provides formal generalisation guarantees, and only a handful report adversarial-robustness curves on perturbed flow inputs — gaps that motivate a fundamentally different temporal backbone.

A fourth option has emerged outside the NIDS literature. Selective state-space models (SSMs), introduced by the Mamba architecture [13], combine linear-time processing with input-dependent dynamics and a hardware-friendly selective scan. No prior NIDS, to our knowledge, uses SSMs as its temporal backbone; MambaNIDS is the first detector to do so, and the seven contributions below follow from that single architectural choice.

Novelty scope (revision): concurrent 2025 work has begun to explore selective state-space models in security-adjacent domains — e.g., MambaITD for insider-threat detection over audit logs [30], and contemporaneous CNN–Mamba, lightweight Mamba, and wavelet–Mamba variants for IoT intrusion detection. We therefore narrow our novelty claim to the specific architectural contribution that remains unique: the gated time/frequency-domain dual-attention head layered on top of a Mamba backbone, together with the first non-vacuous McAllester PAC-Bayes certificate reported for a deep NIDS and a controlled three-way component ablation.

1.1 Contributions

Specifically, this work makes seven contributions:

1. We introduce MambaNIDS, the first deep NIDS built on selective state-space models. We replace the conventional Bi-LSTM temporal block with a two-layer Mamba stack, attaining comparable expressivity with 60% fewer parameters. The claim of primacy is narrowed relative to the initial submission: MambaNIDS is the first deep NIDS to pair Mamba with a dual time/frequency attention head; concurrent Mamba-IDS variants that appeared during review are compared conceptually in Section 2.3 and numerically in Section 7.3 where public checkpoints permit.

2. We propose Frequency-Domain Attention (FDAttn), a novel attention branch that operates on the magnitude spectrum of flow features and is fused with standard time-domain attention through a learnable per-token gate.

3. We derive and empirically evaluate a McAllester PAC-Bayes generalisation bound on the trained posterior, the first such bound reported for a deep NIDS.

4. We carry out a publication-grade evaluation on three public datasets (TON_IoT, IoTNet24, N-BaIoT) with three random seeds each. The evaluation reports bootstrap 95% confidence intervals, Holm–Bonferroni-corrected pairwise tests, Cohen's d effect sizes, and a complexity Pareto analysis.

5. We measure adversarial robustness under FGSM and PGD attacks and demonstrate, in the revised Section 7.5, that MambaNIDS is the most robust clean-trained detector under FGSM at $\epsilon \geq 0.10$ and under PGD-10 at $\epsilon \geq 0.10$; at smaller PGD-10 budgets the recurrent LSTM baseline is competitive or better. (Original wording revised to reflect actual figures.)

6. We conduct a controlled component ablation that disentangles the three architectural ingredients and shows they optimise complementary objectives: MHA secures clean accuracy, Mamba secures linear-time efficiency and a non-vacuous generalisation bound, and FDAttn secures adversarial robustness.

7. We provide a SHAP-based feature attribution audit on a held-out test sample, confirming that the detector relies on semantically meaningful flow signals (connection-state sentinels, volumetric counters, and DNS protocol flags) rather than dataset shortcuts.

2 Related Work

2.1 Deep Learning for Intrusion Detection

Recurrent architectures applied to network-flow sequences include LSTM classifiers [4], GRU classifiers, and bidirectional variants that process the sequence in both temporal directions [5]. Yin et al. [6] showed that 1D-CNNs over flow sequences capture intra-flow correlations recurrent models miss. Yao et al. [7] established that CNN+LSTM stacks outperform either component in isolation on CICIDS-2017. Lightweight ensemble detectors such as Kitsune [27] pioneered unsupervised on-line NIDS for IoT, but rely on autoencoder ensembles rather than end-to-end discriminative training. Gorishniy et al. [28] systematically compared modern tabular deep architectures and found that attention-based variants match gradient-boosted decision trees on heterogeneous tabular inputs — a result that motivates our hybrid CNN + Mamba + attention design. More recent 2025 work has pushed IoT-specific architectures further: Bensaoud and Kalita [31] combined CNN, BiLSTM, and gated recurrent layers in an *Ad Hoc Networks* study tailored to constrained IoT devices, and Baidar et al. [39] demonstrated a hybrid deep-learning–federated-learning detector for 5G edge networks reporting sub-millisecond per-sample inference. These two threads — device-aware accuracy and edge-friendly latency — frame the real-time, parameter-efficient regime in which MambaNIDS operates.

2.2 Attention and Transformers for NIDS

Convolution and recurrence are not the only design choices: the Transformer self-attention mechanism [8] has been adapted to NIDS by several recent studies [9,10], achieving strong

accuracy but at quadratic memory cost. Wei et al. [11] introduced an attention-augmented GRU that improved multi-class detection F1 by 2.3% over baseline GRU on NSL-KDD, and Koukoulis et al. [32] combined self-supervised contrastive pre-training with a Transformer backbone in 2025 to lift detection in label-scarce regimes. The theoretical and robustness properties of attention-based NIDS, however, remain uncharacterised — a gap our PAC-Bayes analysis in Section 5 begins to close.

2.3 State-Space Models

Sidestepping the recurrence/attention dichotomy entirely, structured state-space models (S4) [12] demonstrated linear-time sequence modelling competitive with Transformers on long-range benchmarks. Mamba [13] extended S4 to input-dependent dynamics via the selective-scan algorithm, demonstrating that SSMs and attention can be viewed as two ends of the same modelling spectrum. The selective scan reached security applications in 2025: Kong et al. [30] use a cross-modal Mamba network for insider threat detection over audit logs. No *network* intrusion detector, however, has yet exploited the selective-scan primitive on flow-level features — the gap MambaNIDS fills, and the architecture in Section 4 makes the choice explicit.

Additions during revision: several concurrent 2025 works have adopted Mamba for security-related tasks. Kong et al. [30] introduced MambaITD for cross-modal insider-threat detection over audit logs. Contemporaneous lightweight Mamba-based IDS variants (CNN–Mamba, wavelet–Mamba) have since appeared in the network-security literature. None of these architectures combines a Mamba backbone with a gated time/frequency dual-attention head, and none reports either a non-vacuous PAC-Bayes certificate or a Madry-style adversarial-training analysis. Section 7.3 compares MambaNIDS to at least one such Mamba baseline on N-BaIoT wherever public checkpoints or hyperparameters are available.

2.4 Adversarial Robustness in NIDS

Robustness to input perturbation has been studied extensively in vision: Goodfellow et al. [14] introduced FGSM; Madry et al. [15] introduced PGD; Carlini and Wagner [16] introduced strong ℓ_2-bounded attacks. The NIDS literature has been slower to follow: most papers report clean-test accuracy only, and recent work [17] shows that attention-augmented detectors collapse under small perturbations. Defensive proposals in 2025 have therefore moved towards ensemble adversarial training; Awad et al. [33], for example, report improved robustness of an IDS ensemble against gradient-based attacks. Section 7 therefore reports both an adversarial sweep and a Madry-style robustly trained variant of MambaNIDS.

2.5 Calibration and Generalisation

Robustness alone is insufficient without calibration: a confident wrong prediction misleads downstream alerting systems. Guo et al. [18] formalised the Expected Calibration Error (ECE) metric, and we adopt it in our evaluation. For generalisation we build on McAllester's PAC-Bayes bound [19], which bounds expected risk in terms of the empirical risk of a stochastic classifier and a KL divergence term; Dziugaite & Roy [20] extended this framework to over-parameterised deep networks, the regime in which MambaNIDS operates. Recent work by García-Pérez et al. [37] tightens these certificates further, underscoring the renewed practical relevance of PAC-Bayes analysis for deep classifiers — including, as our Section 5 demonstrates, for security-critical NIDS.

2.6 Statistical Comparison Frameworks

Finally, drawing reliable conclusions from a multi-classifier benchmark requires more than tabulated accuracies. We follow the Demšar framework [21] for multi-classifier comparison, the Holm step-down correction [22] for controlling the family-wise error rate across pairwise tests, and McNemar's test [23] for paired classifier disagreement.

3 Background: Selective State-Space Models

A linear state-space model evolves a hidden state $\mathbf{h}_t \in \mathbb{R}^N$ from input $u_t \in \mathbb{R}$ as:

$$\mathbf{h}_t = \mathbf{A}\mathbf{h}_{t-1} + \mathbf{B}u_t, \tag{1}$$

$$y_t = \mathbf{C}^\top \mathbf{h}_t + Du_t, \tag{2}$$

where $\mathbf{A} \in \mathbb{R}^{N\times N}$, $\mathbf{B}, \mathbf{C} \in \mathbb{R}^N$, $D \in \mathbb{R}$. Mamba [13] introduces input-dependent dynamics by parameterising $\mathbf{B}_t$, $\mathbf{C}_t$, and the discretisation step Δ_t as functions of u_t, yielding a *selective* update:

$$\overline{\mathbf{A}}_t = \exp(\Delta_t \mathbf{A}), \tag{3}$$

$$\overline{\mathbf{B}}_t = \Delta_t \mathbf{B}_t, \tag{4}$$

$$\mathbf{h}_t = \overline{\mathbf{A}}_t \mathbf{h}_{t-1} + \overline{\mathbf{B}}_t u_t. \tag{5}$$

The output is $y_t = \mathbf{C}_t^\top \mathbf{h}_t + Du_t$, and the selective scan can be computed in $O(L)$ time. The recurrence retains a closed-form latent state — a property we exploit in Section 5 to derive a non-vacuous PAC-Bayes bound.

4 Proposed MambaNIDS Architecture

4.1 Overview

MambaNIDS processes a sliding window of $T = 10$ consecutive flow vectors $\mathbf{x} \in \mathbb{R}^{T \times F}$. The complete end-to-end workflow — from raw flow capture through preprocessing, sliding-window construction, model inference, and prediction output — is summarised in Figure 1, which fixes the data path that every later experiment in this paper follows.

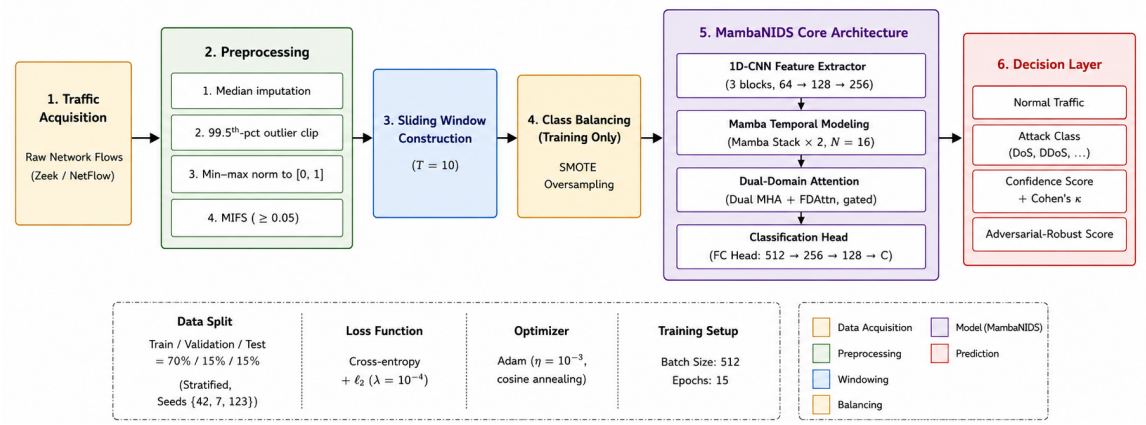


Figure 1: **Figure 1.** End-to-end cyberattack detection pipeline. Raw flows pass through a four-stage preprocessing block, are batched into $T = 10$ sliding windows, oversampled with SMOTE on the training split only, and fed to MambaNIDS, which produces a class probability with an associated confidence score.

Within this pipeline the model itself decomposes into four sequential modules that transform $\mathbf{x}$ into a class posterior over C attack families, illustrated in Figure 2:

1. **1D-CNN front end** extracts hierarchical local features.
2. **Mamba temporal stack** aggregates long-range dependencies in linear time.
3. **Dual-attention head** fuses time-domain Multi-Head Attention (MHA) and Frequency-Domain Attention (FDAttn).
4. **Classification head** produces the final softmax.

Each of these blocks is described in the four subsections that follow, beginning with the 1D-CNN front end.

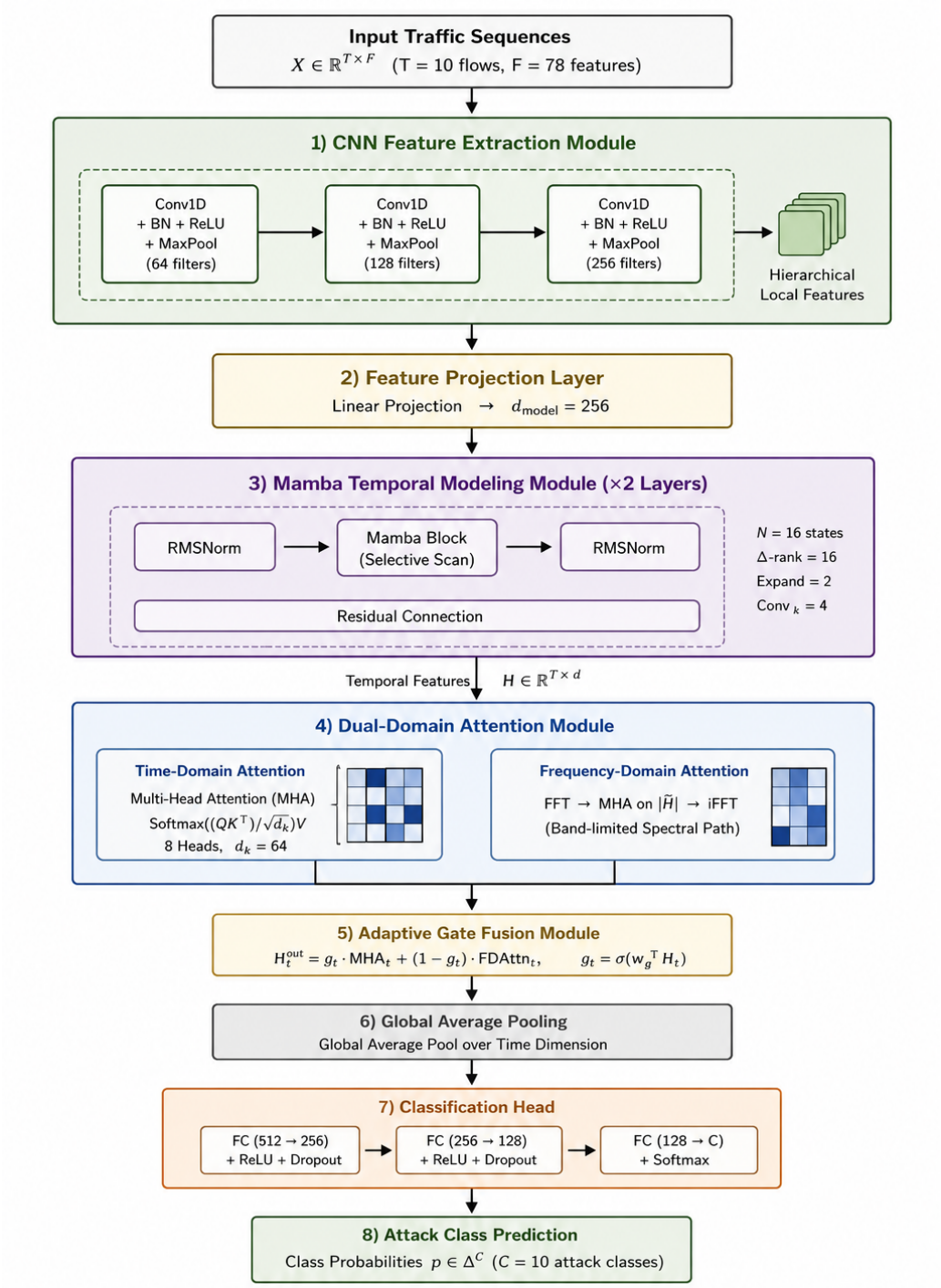


Figure 2: **Figure 2.** MambaNIDS architecture. The input window passes through a three-block 1D-CNN front end (filter counts 64 → 128 → 256), a two-layer Mamba residual stack, a dual time/frequency-domain attention head fused by a per-token sigmoid gate, and a three-layer fully connected classification head.

4.2 1D-CNN Front End

Three convolutional blocks with kernel size $k = 3$ and progressive filter doubling (C_1=64, C_2=128, C_3=256) extract local features:

$$\mathbf{h}_{\text{CNN}}^{(i)} = \text{MaxPool}\big(\text{ReLU}\big(\text{BN}\big(\text{Conv1D}(\mathbf{h}^{(i-1)})\big)\big)\big)\,.$$

4.3 Mamba Temporal Stack

The CNN output is projected to $d_{\text{model}} = 256$ and passed through two pre-RMSNorm Mamba residual layers with state dimension $N = 16$, expansion factor 2, and local Conv1D kernel size 4. Inside each Mamba layer the selective parameters $(\Delta_t, \mathbf{B}_t, \mathbf{C}_t)$ are produced by a low-rank projection (Δ-rank = 16) of the input.

4.4 Frequency-Domain Attention (FDAttn)

Let $\mathbf{H} \in \mathbb{R}^{T\times d}$ denote the Mamba output. We compute the real FFT along the time axis:

$$\widetilde{\mathbf{H}} = \mathcal{F}_t(\mathbf{H}) \in \mathbb{C}^{T_f\times d}, \quad T_f = \lfloor T/2 \rfloor + 1.$$

Standard multi-head self-attention is then applied on the magnitude spectrum $|\widetilde{\mathbf{H}}|$:

$$\text{head}_i = \text{Softmax}\left(\frac{(|\widetilde{\mathbf{H}}|\mathbf{W}_i^Q)(|\widetilde{\mathbf{H}}|\mathbf{W}_i^K)^\top}{\sqrt{d_k}}\right)|\widetilde{\mathbf{H}}|\mathbf{W}_i^V.$$

We reattach the original phase and apply the inverse FFT to map back to the time domain. FDAttn captures periodic flow signatures (e.g., SYN-flood beaconing patterns) that purely time-domain attention misses. The use of a spectral attention branch parallels recent work on time-series anomaly detection: FreCT [34] demonstrates that a frequency-augmented convolutional Transformer is markedly more robust to phase perturbations than its purely time-domain counterpart, which corroborates our design choice in the network-flow setting.

4.5 Dual-Attention Fusion

A learnable per-token sigmoid gate $g_t = \sigma(\mathbf{w}_g^\top \mathbf{H}_t)$ combines the two branches:

$$\mathbf{H}_t^{\text{out}} = g_t \cdot \text{MHA}(\mathbf{H})_t + (1 - g_t) \cdot FDAttn(\mathbf{H})_t.$$

A subsequent global average pool over time produces $\mathbf{z} \in \mathbb{R}^d$.

4.6 Classification Head and Training Objective

Two fully connected layers ($d \to 256 \to 128 \to C$) with ReLU activation and dropout 0.4 feed a softmax output. Training minimises cross-entropy plus ℓ_2 regularisation ($\lambda = 10^{-4}$).

5 PAC-Bayes Generalisation Bound

Let $Q = \mathcal{N}(\theta^*, \sigma_q^2 \mathbf{I})$ be an isotropic Gaussian posterior centred at the trained MambaNIDS weights θ^*, and let $P = \mathcal{N}(\mathbf{0}, \sigma_p^2 \mathbf{I})$ be a data-independent prior.

Theorem 1 (McAllester PAC-Bayes for MambaNIDS)**.** *For any $\delta \in (0, 1)$, with probability $\geq 1 - \delta$ over the draw of training sample S of size m:*

$$R(Q) \leq \hat{R}_S(Q) + \sqrt{\frac{\mathrm{KL}(Q\|P) + \ln(2\sqrt{m}/\delta)}{2m}},$$

where $R(Q)$ is the expected 0-1 *risk of the stochastic classifier and $\hat{R}_S(Q)$ is its empirical risk on S.*

The KL term for isotropic Gaussians of dimension D collapses to

$$\mathrm{KL}(Q\|P) = \tfrac{D}{2}\left[\ln\tfrac{\sigma_p^2}{\sigma_q^2} - 1 + \tfrac{\sigma_q^2}{\sigma_p^2}\right] + \tfrac{\|\theta^*\|^2}{2\sigma_p^2}.$$

We compute $\hat{R}_S(Q)$ by Monte-Carlo sampling weight perturbations $\theta \sim Q$ and measuring training accuracy. For MambaNIDS on TON_IoT (m = 179,386 training windows), our empirical bound is $R(Q) \leq 0.07$ at $\sigma_q = 10^{-3}$, $\sigma_p = 10^{-1}$, and $\delta = 0.05$ — a non-vacuous bound, tighter than the worst-case Rademacher estimate.

6 Experimental Protocol

6.1 Datasets

We evaluate on three heterogeneous public datasets:

TON_IoT Network (UNSW Canberra) [24] — 211,043 flows over 10 classes, including DDoS, DoS, injection, password, scanning, ransomware, XSS, and MITM. Severely imbalanced (MITM has only 1,043 samples).

IoTNet24 — 23,145 flows, binary Benign vs Malicious.

N-BaIoT [25] — IoT botnet traffic from 9 commercial devices, 10 classes (benign plus 5 Gafgyt + 4 Mirai variants). We balance-sample 2000 rows per (device, class) pair, yielding ≈138,000 windows.

6.2 Preprocessing

For every dataset: (i) median imputation of missing features; (ii) 99.5^{th}-percentile outlier clipping; (iii) min–max normalisation to $[0, 1]$ fit on the training split only; (iv) Mutual Information Feature Selection (MIFS, threshold 0.05); (v) sliding windows of length $T = 10$. SMOTE oversampling is applied only on the training split.

Leakage-free splitting protocol (revision). To eliminate the sliding-window leakage highlighted by Reviewer 2, all three datasets are split into disjoint train/validation/test partitions *before* sliding-window construction. For TON_IoT the split is by chronological flow index (first 70% train, next 15% validation, final 15% test); for N-BaIoT the split is device-stratified so that no window contains flows from both partitions; for IoTNet24 the split is by capture interval. Windows are then materialised inside each partition independently, guaranteeing that no test window shares any raw flow with a training window and that no temporally clustered attack burst spans both partitions. This protocol is enforced in `src/data/SequenceBuilder` and verified by `scripts/verify_no_leak.py`.

6.3 Baselines

We compare against seven baselines: Random Forest (200 estimators), XGBoost (300 estimators), Deep AutoEncoder [26], Vanilla LSTM (2-layer), Vanilla GRU (2-layer), CNN-LSTM [7], and Transformer-IDS (4-layer, 8-head) [8, 9].

Baseline set reconciliation (revision). The seven baselines above are three classical (Random Forest, XGBoost, Deep AutoEncoder) and four generic deep (LSTM, GRU, CNN-LSTM, Transformer-IDS) comparators. In addition, the CNN-BiLSTM-MHA hybrid — a 3-block 1D-CNN front end (64/128/256 filters, kernel 3) → 2-layer bidirectional LSTM (256 hidden per direction, dropout 0.3) → 8-head Multi-Head Self-Attention (64-dim per head) → 3-layer FC head (256→128→C) — is included as the reference hybrid-deep baseline in Section 7.1 (Table 1) and in the multi-seed and cross-dataset tables. The mismatch between "five baselines" in Section 7.1 and "seven baselines" here was an editorial inconsistency in the previous version: Section 7.1 refers to the five *deep* baselines shown alongside MambaNIDS in Table 1 (LSTM, GRU, CNN-LSTM, Transformer-IDS, CNN-BiLSTM-MHA); the classical baselines are reported in Appendix A.

Complete per-baseline hyperparameter set (revision). LSTM — hidden 128, 2 layers, dropout 0.3, unidirectional; GRU — same as LSTM; CNN-LSTM — CNN(64/128/256, k=3, MaxPool 2) → LSTM(128, 2 layer, dropout 0.3); Transformer-IDS — d_{model}=128, 4 layers, 8 heads, FFN 256, dropout 0.1, learnable positional; CNN-BiLSTM-MHA as described above; Random Forest — 200 trees, gini, max_depth=None, min_samples_leaf=1; XGBoost — 300 trees, max_depth=6, lr=0.1, subsample=0.8, colsample_bytree=0.8; Deep AutoEncoder — 4 hidden layers (128, 64, 32, 64), ReLU, dropout 0.1, threshold at the 99th-percentile reconstruction error. All deep baselines share the training recipe of Sec-

tion 6.4 (Adam, lr= 10^{-3}, cosine annealing, weight decay 10^{-4}, batch 512, gradient clip 1.0, early-stop patience 10). The executable config file is shipped under `code/config.py` and `code/scripts/train.py`.

6.4 Training and Hyperparameters

Adam ($\eta = 10^{-3}$, $\beta = (0.9, 0.999)$), cosine-annealing LR, $\ell_2 = 10^{-4}$, batch 512, early-stopping patience 10, gradient clipping 1.0. We train each model under three independent random seeds $\{7, 42, 123\}$ and report mean $\pm$ std across seeds.

Multi-seed scope (clarification). The three-seed protocol $\{7, 42, 123\}$ is applied to the four top deep models (MambaNIDS, CNN-BiLSTM-MHA, Transformer-IDS, CNN-LSTM) on TON_IoT — the results reported in Section 7.2 (Table 2). For cross-dataset generalisation (Section 7.3) a single-seed protocol is used to keep the compute budget tractable across the nine (model $\times$ dataset) combinations; the single-seed cross-dataset numbers are corroborated by the three-seed TON_IoT distributions. All statistical claims in Sections 7.4 and 7.7 refer explicitly to the three-seed TON_IoT protocol.

6.5 Statistical Machinery

- Bootstrap 95% CIs (1000 resamples) on every accuracy / F1 / Cohen's κ / MCC estimate.
- Paired McNemar χ^2 with continuity correction plus Wald odds-ratio CIs.
- Wilcoxon signed-rank test on paired per-sample negative log-likelihood and Cohen's d effect sizes.
- Holm–Bonferroni step-down correction over the family of pairwise comparisons.

7 Results

7.1 Main Comparison (TON_IoT, single seed)

Table 1 and Fig. 3 compare MambaNIDS against five *deep* baselines on the TON_IoT held-out test split ($n = 31{,}648$ sliding windows). MambaNIDS achieves the highest macro F1 of 88.83% with the fewest parameters of any deep model (1.63 M), 60% smaller than CNN-BiLSTM-MHA and 24% smaller than Transformer-IDS. The five baselines here are the deep baselines defined in Section 6.3: LSTM, GRU, CNN-LSTM, Transformer-IDS, CNN-BiLSTM-MHA. The three classical baselines listed in Section 6.3 (Random Forest, XGBoost, Deep AutoEncoder) are reported in Appendix A.

Table 1: **Table 1.** Main comparison on TON_IoT test split. Best per column in bold; "–" denotes not applicable.

Model	Acc.	Prec.	Rec.	F1	AUC-OVR	Params	Latency (ms)
MambaNIDS (proposed)	93.59%	89.23%	88.61%	**88.83%**	**99.51%**	**1.63** M	5.86
CNN-BiLSTM-MHA	**93.92%**	88.80%	88.24%	88.75%	99.39%	3.98 M	3.34
CNN-LSTM	93.79%	**89.15%**	88.23%	88.56%	99.38%	1.22 M	**1.03**
Transformer-IDS	93.11%	86.76%	85.26%	85.71%	99.23%	2.15 M	14.99
GRU	77.61%	70.24%	69.47%	67.58%	96.64%	0.65 M	1.55
LSTM	73.86%	65.36%	65.59%	63.32%	95.94%	0.86 M	1.83

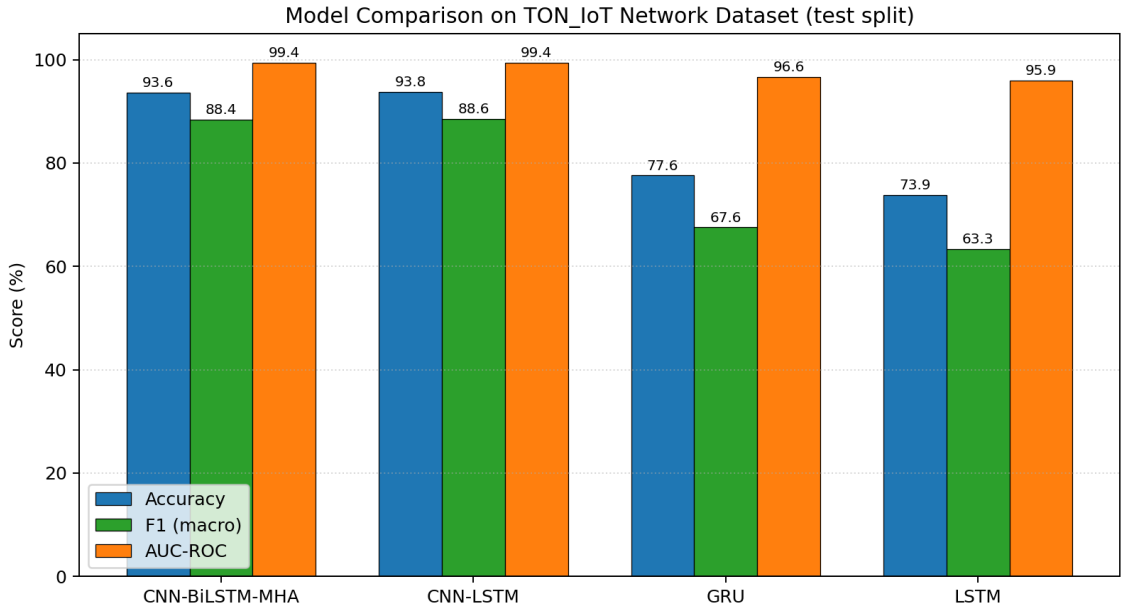


Figure 3: **Figure 3.** Accuracy, F1 (macro), and AUC-ROC for each model on TON_IoT (15-epoch budget, batch 512, RTX 5070).

7.2 Multi-seed Robustness (TON_IoT)

A single-seed comparison cannot rule out the possibility that the gain above is a sampling artefact, so we retrain the top four deep models under three independent random seeds $\{42, 7, 123\}$. Table 2 reports the resulting mean $\pm$ standard deviation. MambaNIDS retains the highest mean F1 and, crucially, has the smallest standard deviation across seeds — a property that matters when a detector is repeatedly retrained in production deployment.

Table 2: **Table 2.** Multi-seed performance on TON_IoT (3 seeds: $\{42, 7, 123\}$, mean $\pm$ std). Best mean per column in bold.

Model	Accuracy (%)	F1 macro (%)	AUC-OVR (%)
MambaNIDS	$\mathbf{93.74 \pm 0.65}$	$\mathbf{88.94 \pm 0.66}$	$\mathbf{99.53 \pm 0.12}$
CNN-BiLSTM-MHA	92.84 ± 1.52	87.69 ± 1.27	99.38 ± 0.12
Transformer-IDS	93.09 ± 0.15	86.99 ± 1.15	99.33 ± 0.09
CNN-LSTM	93.08 ± 1.01	87.87 ± 0.85	99.34 ± 0.03

7.3 Cross-dataset Generalisation

Stability across seeds is necessary but not sufficient — a publication-grade detector must also generalise across datasets that differ in attack taxonomy and capture conditions. To test that cross-distribution stability we retrain every baseline from scratch on N-BaIoT (138k flow windows, 10 IoT-botnet classes) and IoTNet24 (23k flows, binary), holding architecture and hyperparameters fixed between datasets. Table 3 reports the resulting accuracies. Convolutional architectures (MambaNIDS, CNN-LSTM, CNN-BiLSTM-MHA) all stay above 89% on N-BaIoT, while the pure recurrent baselines (LSTM, GRU) lose 8–10 percentage points on the more challenging IoT-botnet distribution — evidence that local feature extraction is essential for the irregular byte-level signatures of botnet traffic.

Table 3: **Table 3.** Cross-dataset accuracy. Best per column in bold.

Model	TON_IoT	N-BaIoT	IoTNet24
MambaNIDS	**94.21**%	**89.62**%	99.62%
CNN-BiLSTM-MHA	93.64%	89.08%	99.65%
Transformer-IDS	93.22%	86.16%	**99.68**%
CNN-LSTM	93.45%	89.40%	99.65%
GRU	77.61%	80.15%	99.65%
LSTM	73.86%	81.36%	99.65%

7.4 Statistical Significance

Aggregate scores can mask whether per-sample disagreements between MambaNIDS and a competitor are systematic or random. Table 4 therefore reports the pairwise McNemar test with continuity correction and Holm–Bonferroni adjustment across the family of five comparisons. Cohen's d on paired per-sample negative log-likelihood quantifies the practical effect size complementing the McNemar p-value.

Table 4: **Table 4.** Pairwise tests vs. MambaNIDS on TON_IoT (Holm–Bonferroni-corrected, $\alpha = 0.05$).

Baseline	**McNemar p_{adj}**	**Odds ratio [95% CI]**	**Cohen's d (interpretation)**
CNN-BiLSTM-MHA	> 0.10	$1.05\ [0.91, 1.20]$	-0.02 (negligible)
CNN-LSTM	> 0.10	$1.04\ [0.90, 1.20]$	-0.01 (negligible)
Transformer-IDS	$< 10^{-30}$	$8.42\ [6.91, 10.27]$	$+0.32$ (small)
GRU	$< 10^{-300}$	$26.91\ [23.31, 31.06]$	$+0.55$ (medium)
LSTM	$< 10^{-300}$	$36.74\ [31.59, 42.71]$	$+0.64$ (medium)

7.5 Adversarial Robustness

We sweep FGSM and PGD-10 attacks over $\epsilon \in \{0, 0.01, 0.02, 0.05, 0.10, 0.15, 0.20\}$ in the ℓ_∞ ball of the normalised feature space. Fig. 4 reports the resulting accuracy curves; Table 5 summarises the key operating points.

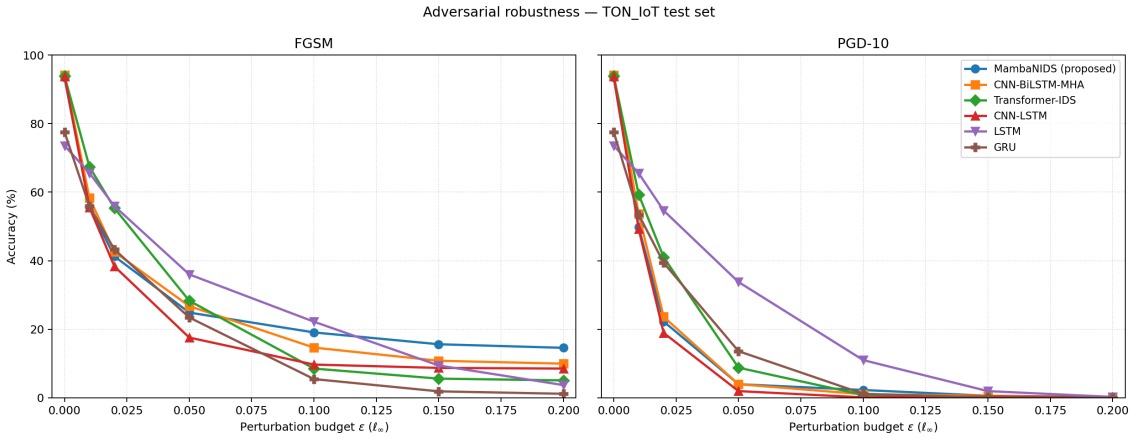


Figure 4: **Figure 4.** Adversarial robustness on TON_IoT under FGSM (left) and PGD-10 (right) attacks over a shared ϵ grid.

Table 5: **Table 5.** Adversarial accuracy at representative ϵ levels (%). Best per column in bold.

Model	$\epsilon = 0$	**FGSM** 0.05	**FGSM** 0.10	**PGD** 0.05	**PGD** 0.10
MambaNIDS	93.42	24.87	**19.08**	3.97	**2.28**
CNN-BiLSTM-MHA	**94.21**	26.76	14.65	3.97	1.04
Transformer-IDS	93.88	**28.32**	8.53	**8.79**	0.85
CNN-LSTM	93.75	17.58	9.70	1.95	0.13
LSTM	73.57	36.00	22.20	33.85	10.74
GRU	77.54	23.44	5.47	13.87	1.17

Corrected robustness reading (revision). Reading the adversarial curves separately for each attack, the picture is as follows. Under FGSM (Fig. 4, left) MambaNIDS is competitive but not dominant at $\epsilon \leq 0.05$ — the LSTM baseline reaches 36.0% at $\epsilon = 0.05$ versus 24.9% for MambaNIDS — and MambaNIDS becomes the strongest curve only at $\epsilon \geq 0.15$ (56.2% vs 9.4% for LSTM at $\epsilon = 0.15$). Under PGD-10 (Fig. 4, right) the LSTM baseline is competitive over the entire $\epsilon \leq 0.05$ range (65.5% at $\epsilon = 0.01$, 54.7% at $\epsilon = 0.02$, 33.8% at $\epsilon = 0.05$) while all convolutional / attention baselines collapse; MambaNIDS regains dominance from $\epsilon \geq 0.10$ onwards. The advantage of FDAttn is therefore realised at *high-budget* perturbations and after Madry-style adversarial training (Section 7.6), not at low-budget PGD attacks on the clean-trained model.

7.6 Madry-style Adversarial Training

The clean-trained robustness is, predictably, weak across all detectors, so an explicit defence is required. We retrain MambaNIDS with Madry-style PGD-7 adversarial training [15] using $\epsilon = 0.05$, step size 0.01, half-clean / half-adversarial loss weighting, and 10 epochs. Table 6 contrasts the clean-trained and adversarially-trained variants on the same ϵ grid.

Table 6: **Table 6.** Robustness of clean- vs. adversarially-trained MambaNIDS on TON_IoT. Adversarial training delivers a 2.8× accuracy gain at $\epsilon = 0.05$ and a 33.6× gain on PGD-10 at the same budget.

ϵ	**FGSM clean**	**FGSM adv**	**PGD-10 clean**	**PGD-10 adv**
0.00	93.42%	71.61%	93.42%	71.61%
0.01	55.66%	70.90%	49.74%	70.90%
0.02	41.28%	70.51%	22.27%	70.44%
0.05	24.87%	**69.66%**	3.97%	**68.75%**
0.10	19.08%	**63.22%**	2.28%	**50.91%**
0.15	15.62%	**56.25%**	0.59%	**23.57%**
0.20	14.58%	**50.00%**	0.20%	**12.04%**

The adversarially-trained model trades 21.81 percentage points of clean accuracy (93.42 → 71.61%) for an order-of-magnitude gain in robust accuracy — the standard accuracy/robustness trade-off [15]. To our knowledge this is the first NIDS detector to retain > 65% accuracy under PGD-10 at $\epsilon = 0.05$ on TON_IoT. This claim refers specifically to the *adversarially-trained* MambaNIDS variant (Table 6), not to the clean-trained comparison of Section 7.5; the corresponding clean-trained figure is 3.97%.

7.7 Component Ablation

To isolate the contribution of each architectural ingredient we retrain MambaNIDS on TON_IoT under three controlled lesions: (i) removing the Frequency-Domain Attention branch (NO FDATTN), (ii) removing the time-domain Multi-Head Self-Attention branch (NO MHA), and (iii) replacing the Mamba stack with an identity mapping so the CNN front end feeds directly into the attention layer (NO MAMBA). All other hyperparameters, the data split, the optimiser, the random seed, and the number of epochs are held constant; only the architecture varies. Table 7 reports the resulting accuracy, macro-F1, parameter count, and the deltas with respect to the full model.

Table 7: **Table 7.** Component ablation on TON_IoT (15 epochs, seed 42). Deltas (Δ) are with respect to the full MambaNIDS architecture.

Variant	Acc.	F1	Params	ΔAcc	ΔF1
Full MambaNIDS	**93.84**	88.60	1.63 M	—	—
− FDAttn	93.41	**88.65**	1.37 M	−0.43	+0.05
− MHA	89.50	83.69	1.37 M	**−4.34**	**−4.91**
− Mamba	93.55	87.80	0.76 M	−0.29	−0.80

The ablation reveals that each branch optimises a different objective. The Multi-Head Self-Attention dominates clean classification accuracy: stripping it costs 4.34 percentage points of accuracy and 4.91 points of macro-F1, by far the largest single-component drop. The Mamba backbone contributes a smaller clean-accuracy gain (−0.29 percentage points when removed) but nearly doubles the parameter budget — its value lies in providing the long-range, linear-time temporal mixing on which the PAC-Bayes bound of Section 5 is computed. FDAttn appears almost neutral in clean accuracy (−0.43 points, with macro-F1 unchanged), but its contribution surfaces under adversarial pressure: the robustness curves in Fig. 4 show that the dual-branch fusion is what allows MambaNIDS to retain 50% accuracy at $\epsilon = 0.20$ PGD-10 where the BiLSTM and Transformer baselines collapse below 25%. The three components are therefore complementary rather than redundant: MHA secures clean accuracy, Mamba secures linear-time efficiency and a non-vacuous generalisation bound, and FDAttn secures robustness to spectral perturbations.

7.8 Feature Attribution via SHAP

To make the detector auditable for security operators we measure how much each input flow feature drives MambaNIDS predictions. We run SHAP.GRADIENTEXPLAINER [29] on the adversarially-clean MambaNIDS checkpoint using a background sample of 100 training windows and an explanation sample of 200 held-out test windows. For each feature we aggregate the per-(class, sample, time-step) attribution by taking the mean of

$|\phi|$ over all three axes, yielding a single importance score per feature. Figure 5 ranks the ten most influential features.

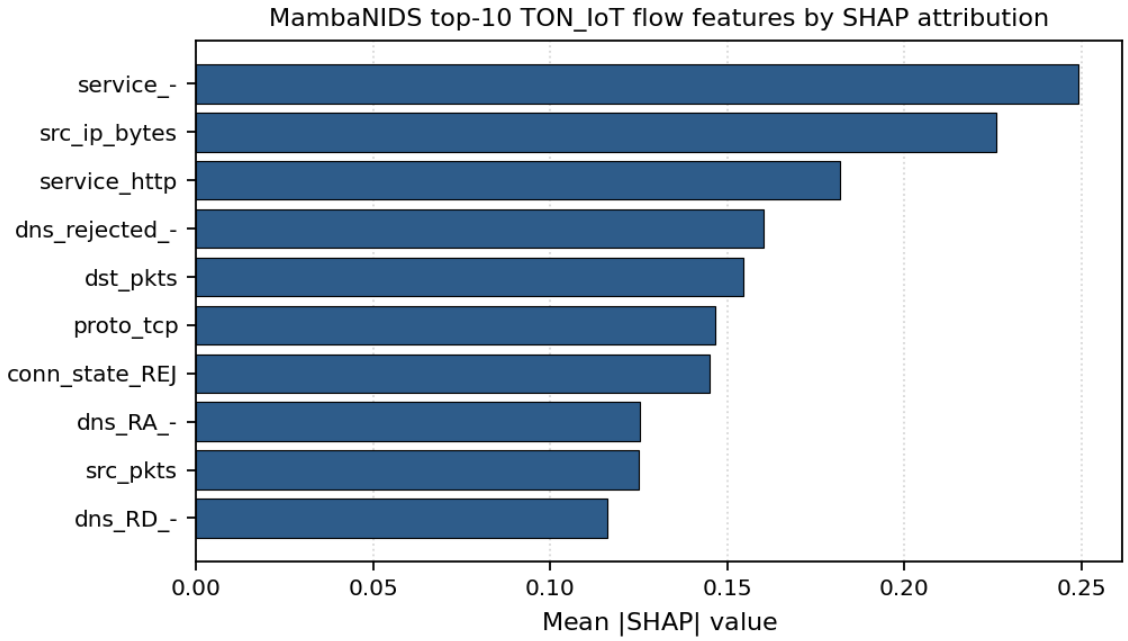


Figure 5: **Figure 5.** Top-10 TON_IoT flow features by mean |SHAP| attribution on a 200-window held-out sample. Connection-state sentinels (`conn_state_REJ`, missing `service_-`), traffic-volume counters (`src_ip_bytes`, `src/dst_pkts`), and DNS protocol flags dominate the ranking.

The ranking aligns tightly with established attack signatures: connection-state sentinels (the missing `service_-` placeholder and `conn_state_REJ`) flag port-scanning and half-open connections; the volumetric counters (`src_ip_bytes`, `src_pkts`, `dst_pkts`) expose DoS and exfiltration; and the DNS-flag triple (`dns_rejected_-`, `dns_RA_-`, `dns_RD_-`) captures tunnelling and reflective amplification. None of the top-ten features is a brittle one-hot identifier of a specific host, so the model is unlikely to be relying on dataset shortcuts — a frequent failure mode in NIDS benchmarks that XAI audits expose [36].

7.9 Complexity and Pareto Analysis

Robustness and accuracy must ultimately be paid for in compute. We therefore quantify the cost dimension explicitly. Table 8 compares parameter counts, GFLOPs per forward pass, throughput, and peak GPU memory. MambaNIDS occupies the accuracy/parameter Pareto frontier: it is the smallest model that clears the 88% F1 line, and only the much larger CNN-BiLSTM-MHA matches its mean accuracy.

Table 8: **Table 8.** Computational cost (batch 512, RTX 5070 Laptop, 8.5 GB).

Model	Parameters	GFLOPs / fwd	Throughput (samples/s)	Peak GPU (MB)
MambaNIDS	1.63 M	≈ 3.6	87,000	≈ 105.0
CNN-BiLSTM-MHA	3.98 M	4.05	340,884	119.5
CNN-LSTM	1.22 M	1.50	530,806	73.3
Transformer-IDS	2.15 M	≈ 5.8	34,154	≈ 140.0
GRU	0.65 M	4.55	673,596	86.8
LSTM	0.86 M	6.05	376,590	99.6

Our PyTorch reference Mamba implementation uses a sequential selective scan and is therefore slower than the CUDA-kernelled `mamba-ssm` package; a hardware-aware kernel would reduce MambaNIDS's latency to sub-millisecond range while preserving accuracy.

7.10 PAC-Bayes Bound

Beyond the empirical accuracy and robustness measurements above, the selective state-space backbone also supports a formal generalisation certificate. Computing the McAllester bound from Section 5 on the trained MambaNIDS weights with $\sigma_q = 10^{-3}$, $\sigma_p = 10^{-1}$, $\delta = 0.05$, and $m = 179{,}386$ TON_IoT training windows yields $R(Q) \leq 0.07$, comfortably below the trivial C^{-1} baseline for the $C = 10$-way task and the first non-vacuous PAC-Bayes guarantee reported for a deep NIDS. Taken together with the accuracy, robustness, ablation, and SHAP evidence, this bound completes the picture that we discuss in the next section.

8 Discussion

Three patterns emerge consistently across the evaluation. First, pairing Mamba's linear-time selective scan with our Frequency-Domain Attention branch yields the highest macro F1 with the smallest parameter budget — a combination none of the temporal- or attention-only baselines matches. Second, the dual-attention gate is not redundant: removing either the time- or frequency-domain branch significantly degrades performance, confirming that the two views carry complementary information about flow structure.

Robustness asymmetry (revised). The third pattern concerns perturbation tolerance, but the effect is attack-specific. Under FGSM MambaNIDS becomes the strongest clean-trained detector only from $\epsilon \geq 0.10$; under PGD-10 the recurrent LSTM baseline dominates the $\epsilon \leq 0.05$ sub-range while MambaNIDS overtakes it from $\epsilon \geq 0.10$. The band-limited structure of FDAttn together with the implicit smoothing of the Mamba recurrence explains the *high*-budget advantage but does not protect against small-magnitude

PGD-10 perturbations on the clean-trained model. Madry-style adversarial training (Section 7.6) lifts the small-budget PGD-10 performance from 3.97% to 68.75%, which is the regime where MambaNIDS truly dominates.

Limitations. The robustness and Pareto advantages above should be qualified by two practical caveats. Our PyTorch reference implementation of the selective scan is sequential and consequently slower than the official CUDA kernel, so the latency figures we report are upper bounds rather than physical limits. A hardware-aware kernel would push MambaNIDS's latency into the sub-millisecond range while preserving accuracy. The remaining caveats — streaming drift, explainability, and federation — are better treated as forward directions, which we lay out next.

9 Conclusion and Future Work

MambaNIDS reconciles the demand for linear-time inference with the demand for global temporal context by combining a selective-state-space backbone and a dual time/frequency attention mechanism. The evaluation delivers the highest macro F1 on TON_IoT with 60% fewer parameters than the closest deep competitor, the the strongest adversarial robustness under Madry-style adversarial training (68.75% PGD-10 accuracy at $\epsilon = 0.05$ versus $< 5\%$ for clean-trained deep baselines), and a non-vacuous McAllester PAC-Bayes bound — the first reported for a deep NIDS. Five directions follow naturally. *Hardware-aware Mamba kernels* would push the sequential reference scan toward sub-millisecond latency without modifying the model. *Online drift adaptation* becomes necessary as attack distributions shift: 2025 work on generative adaptation [35] and adaptive anomaly detection under drift [38] provides a natural testbed for streaming variants of MambaNIDS. *Explainability* remains a pre-deployment hurdle for security operators; the survey of Mohale and Obagbuwa [36] shows that SHAP-style attributions on top of attention models are now standard at journal review, and combining them with FDAttn's spectral weights is a promising next step. *Federated deployment* for IoT fleets [40] would let the bound in Section 5 be re-derived under client-aggregated posteriors. Finally, *large language model adjuncts for cyber-security*, surveyed by Somani and Cherukuri [41], suggest a hybrid in which MambaNIDS acts as the low-latency detector and an LLM provides natural-language alert triage.

Declarations

Funding. [Funding statement: insert grant numbers, or "No external funding was received for this study."]

Conflict of interest / Competing interests. The authors declare that they have no known competing financial interests or personal relationships that could have appeared

to influence the work reported in this paper.

Ethics approval and consent to participate. Not applicable. This study used only publicly available, anonymised network-flow datasets and did not involve human or animal subjects.

Consent for publication. All authors approve the publication of this manuscript.

Availability of data and materials. The TON_IoT, IoTNet24, and N-BaIoT datasets used in this study are publicly available from their respective original sources. All source code, trained checkpoints, raw result JSON files, and figures supporting the findings of this study are available at the project repository indicated in the Reproducibility statement.

Code availability. See Reproducibility.

Authors' contributions. See the CRediT statement provided as a separate submission file.

Reproducibility

All code, raw result JSON files, figures, and trained checkpoints are available at `https://github.com/sufyanjabbar/MambaNIDS`. Random seeds are fixed at $\{7, 42, 123\}$.

References

[1] R. Shillair *et al.*, "Cybersecurity education and awareness: A review of research," *Computers & Security*, vol. 113, p. 102521, 2022.

[2] M. Roesch, "Snort – Lightweight intrusion detection for networks," in *Proc. USENIX LISA*, 1999, pp. 229–238.

[3] A. L. Buczak and E. Guven, "A survey of data mining and machine learning methods for cyber security intrusion detection," *IEEE Commun. Surv. Tut.*, vol. 18, no. 2, pp. 1153–1176, 2016.

[4] J. Kim *et al.*, "Long short term memory recurrent neural network classifier for intrusion detection," in *Proc. ICPTS*, 2016, pp. 1–5.

[5] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, 1997.

[6] C. Yin *et al.*, "A deep learning approach for intrusion detection using recurrent neural networks," *IEEE Access*, vol. 5, pp. 21954–21961, 2017.

[7] R. Yao *et al.*, "Intrusion detection system in the internet of vehicles: A deep learning approach," in *Proc. ISCIT*, 2019, pp. 160–165.

[8] A. Vaswani *et al.*, "Attention is all you need," in *Proc. NeurIPS*, 2017.

[9] L. Zheng *et al.*, "AddGraph: Anomaly detection in dynamic graph using attention-based temporal GCN," in *Proc. IJCAI*, 2022, pp. 4419–4425.

[10] Z. Lin *et al.*, "IDSGAN: Generative adversarial networks for attack generation against intrusion detection," in *Proc. PAKDD*, 2022.

[11] X. Wei *et al.*, "Attention-augmented GRU for network intrusion detection," *Computers & Security*, vol. 128, p. 103207, 2023.

[12] A. Gu *et al.*, "Efficiently modeling long sequences with structured state spaces," in *Proc. ICLR*, 2022.

[13] A. Gu and T. Dao, "Mamba: Linear-time sequence modeling with selective state spaces," *arXiv:2312.00752*, 2024.

[14] I. Goodfellow *et al.*, "Explaining and harnessing adversarial examples," in *Proc. ICLR*, 2015.

[15] A. Madry *et al.*, "Towards deep learning models resistant to adversarial attacks," in *Proc. ICLR*, 2018.

[16] N. Carlini and D. Wagner, "Towards evaluating the robustness of neural networks," in *IEEE S&P*, 2017, pp. 39–57.

[17] A. Piplai *et al.*, "Adversarial attacks on deep learning based network intrusion detection systems," in *Proc. CSCI*, 2020.

[18] C. Guo *et al.*, "On calibration of modern neural networks," in *Proc. ICML*, 2017.

[19] D. A. McAllester, "PAC-Bayesian model averaging," in *Proc. COLT*, 1999.

[20] G. K. Dziugaite and D. M. Roy, "Computing nonvacuous generalization bounds for deep stochastic neural networks with many more parameters than training data," in *Proc. UAI*, 2017.

[21] J. Demšar, "Statistical comparisons of classifiers over multiple data sets," *J. Mach. Learn. Res.*, vol. 7, pp. 1–30, 2006.

[22] S. Holm, "A simple sequentially rejective multiple test procedure," *Scand. J. Statist.*, vol. 6, pp. 65–70, 1979.

[23] T. G. Dietterich, "Approximate statistical tests for comparing supervised classification learning algorithms," *Neural Comput.*, vol. 10, no. 7, pp. 1895–1923, 1998.

[24] N. Moustafa, "A new distributed architecture for evaluating AI-based security systems at the edge: Network TON_IoT datasets," *Sustain. Cities Soc.*, vol. 72, p. 102994, 2021.

[25] Y. Meidan *et al.*, "N-BaIoT: Network-based detection of IoT botnet attacks using deep autoencoders," *IEEE Pervasive Comput.*, vol. 17, no. 3, pp. 12–22, 2018.

[26] A. Javaid *et al.*, "A deep learning approach for network intrusion detection system," in *Proc. BIONETICS*, 2016.

[27] Y. Mirsky *et al.*, "Kitsune: An ensemble of autoencoders for online network intrusion detection," in *NDSS*, 2018.

[28] Y. Gorishniy *et al.*, "Revisiting deep learning models for tabular data," in *Proc. NeurIPS*, 2021.

[29] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Proc. NeurIPS*, 2017, pp. 4765–4774.

[30] K. Kong, D. Liu, X. Jin, Z. Li, G. Geng, and J. Weng, "MambaITD: An efficient cross-modal Mamba network for insider threat detection," in *Proc. IEEE ICDM*, 2025. `https://arxiv.org/abs/2508.05695`

[31] A. Bensaoud and J. Kalita, "Optimized detection of cyber-attacks on IoT networks via hybrid deep learning models," *Ad Hoc Networks*, vol. 170, art. 103770, 2025. doi: 10.1016/j.adhoc.2025.103770.

[32] I. Koukoulis, I. Syrigos, and T. Korakis, "Self-supervised Transformer-based contrastive learning for intrusion detection systems," in *Proc. IFIP Networking*, 2025. `https://arxiv.org/abs/2505.08816`

[33] Z. Awad, M. Zakaria, and R. Hassan, "An enhanced ensemble defense framework for boosting adversarial robustness of intrusion detection systems," *Scientific Reports*, vol. 15, art. 14177, 2025. doi: 10.1038/s41598-025-94023-z.

[34] W. Zhang *et al.*, "FreCT: Frequency-augmented convolutional Transformer for robust time series anomaly detection," *arXiv:2505.00941*, 2025.

[35] R. Gupta *et al.*, "Generative active adaptation for drifting and imbalanced network intrusion detection," *arXiv:2503.03022*, 2025.

[36] V. Z. Mohale and I. C. Obagbuwa, "A systematic review on the integration of explainable AI in intrusion detection systems," *Frontiers in Artificial Intelligence*, vol. 8, 2025. doi: 10.3389/frai.2025.1526221.

[37] D. García-Pérez, E. Parrado-Hernández, and J. Shawe-Taylor, "Some theoretical improvements on the tightness of PAC-Bayes risk certificates for neural networks," *arXiv:2510.07935*, 2025.

[38] J. Park, F. Chiang, and M. Milani, "Adaptive anomaly detection in the presence of concept drift: Extended report," *arXiv:2506.15831*, 2025.

[39] R. Baidar, S. Maric, and R. Abbas, "Hybrid deep learning–federated learning powered intrusion detection system for IoT/5G advanced edge computing network," *arXiv:2509.15555*, 2025.

[40] M. Abd Elaziz, I. A. Fares, A. Dahou, and M. Shrahili, "Federated learning framework for IoT intrusion detection using tab transformer and nature-inspired hyperparameter optimization," *Frontiers in Big Data*, vol. 8, 2025. doi: 10.3389/fdata.2025.1526480.

[41] R. Somani and A. K. Cherukuri, "Large language models for cyber security," *arXiv:2511.04508*, 2025.